THE NATIONAL UNIVERSITY OF LESOTHO
Department of Mathematics and Computer Science · Faculty of Science and Technology
CS3520 Computer Organisation and Architecture · AY 2026/2027

LIMO Group Project · Milestone M1 Submission

LIMO Instruction Set Architecture

Team	RISC-Management
Lab group	A,B,C
Release tag	m1
Commit	
Due	Wed 14 Oct 2026, 23:59
Page limit	6 pages
Repository	https:/lgithub.com/Abuti-Bernard/llmo-RISC-Management
Live site	https://Abuti-Bernard.github.io/limo-RISC-Management/
Assessed	Part I: ISA specification A1–A5 (40) and design-decision log A6 (20); defended individually at D1 (40).

#	Member	Student no.	GitHub
1	Mzinyathi Ndaba	202400160	Abuti-Bernard
2	Hlabaki Ncheke	202321245	hlabaki
3	Ntaba Lebamang	202408581	leba-bloom
4	'Neheng Mokoali	202400462	Neheng-Mokoali
5	Mthimkhulu Ntimo	202404616	muthintimo-afk
6	Thembezile Mothoalo	202201984	vinwrld9
7			

1 Team Charter and Set-up

Project board	https://github.com/users/Abuti-Bernard/projects/2
Meeting rhythm	Any weekday 7pm, touchups on WhatsApp group
Decision rule	Vote
Definition of done	Everything documented
Branch and PR rules	https://github.com/Abuti-Bernard/limo-RISC-Management/settings/rules
When a member is absent or not contributing	Asked to contribute or given more work

2 ISA Specification

2.1 Overview

	Specification	Rationale

	Specification	Rationale
Architecture style	32-bit load–store; 3-address architecture	8 registers are too small and memory access is frequent, 16 registers has chances of stack traffic. 32 registers offer flexibility without saving to memory everytime.
Register bank	27 GPRs and 5 SPRs	Sephetho0-7 for arguments and return values, nakoana1-7 for temporary storage, letsete0-11 for long lived values. Mokou, sephira, letsibolo, khoele and lehe needed for stack, control flow and zero.
Addressing modes	Register, immediate, base, PC-relative, register indirect	For sfast arithmetic and logical operations, operations involving constants more efficient,accessing data at specific memory locations, conditional branching and control flow without requiring absolute addresses.
Memory, endianness, alignment	Byte-addressed, little-endian, word-aligned; 2^{32} bytes=4GB, little endian, 4 bytes	Little-endian byte ordering stores the least significant byte at the lowest address, 32 bits offer a large address space
Instruction formats	R,I,S,B	To allow processor to easily perform arithmetic operations
Departures from RV32I	None	One of the goals of the assignment was something closer to the real RV32 1, it did not seem necessary to depart from it

2.2 Register File

Number	Sesotho name	Alias	Type	Purpose and convention
x0	lehe	zero	SPR	Hard-wired to 0; writes are ignored
x1	letsibolo	ra	SPR	Return address
x2	mokou	sp	SPR	Stack pointer
x3	sephara	gp	SPR	Global pointer
x4	khoele	tp	SPR	Thread pointer
x10-x17	sephetho0-sephetho7	a0-a6	GPR	Function argument
x5-x7, x28-x31	nakoana1-nakoana7	t0-t6	GPR	Temporary storage
X8-x9, x18-x27	letsete0-letsete11	s0-s11	GPR	Store values across function calls
PC	-	–	SPR	Address of the current instruction

2.3 Instruction Formats

	31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode

	31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode

Immediate ranges: -2048 to 2047

2.4 Instruction Set and Sesotho Mnemonics

Class	Mnemonic	Meaning	Syntax	Operation	Fmt	RV32I
Arithmetic	kopanya	add	kopanya rd, rs1, rs 2	R[rd] ← R[rs1] + R[rs2]	R	add
Arithmetic	tlosa	subtract	tlosa rd,rs1,rs2	R[rd] ← R[rs1] + R[rs2]	R	sub
Arithmetic	eketsa	add immedi ate	eketsa rd, rs1,rs2	R[rd] ← R[rs1]	I	addi
Arithmetic	bofahh	load immedi ate	bofahh rd, k	R[rd] ← R[rs1]	I	li
Logic	kapa	or	kapa	R[rd] ← R[rs1] + R[rs2]	R	or
Logic	le	and	le	R[rd] ← R[rs1] + R[rs2]	R	and
Logic	suthumets a	slli	suthumetsa	R[rd] ← R[rs1]	I	slli
Logic	chechisa	mv	chechisa	R[rd] ← R[rs1] + R[rs2]	R	mv
Memory	bofa	lw	bofa	R[rd] ← R[rs1]	I	lw
Memory	boloka	sw	boloka	R[rs1] + R[rs2]	S	sw
Memory	qhoma	jalr	qhoma	R[rd] ← R[rs1]	I	jalr
Branch	lekana	branch if eq ual	lekana	R[rs1] - R[rs2]	B	beq
Branch	ekholo	branch if gr eater than	ekholo	R[rs1] - R[rs2]	B	bge

Lexical rules: case sensitivity, comment marker, labels, register names accepted (number, alias, Sesotho name), decimal and hex immediates, and how apostrophes such as ts' and ch' are handled, and why.

Lexical rules. Uses lowercase letters, a comment marker is '#', x0 to x31. Decimal numbers are written a s they are and hexadecimal must be prefixed with '0x'. The apostrophe shall be treated as part of string.

2.5 Encoding Worked Examples

Instruction	Fmt	Field values	Binary (bit 31 … 0)	Hex
kopanya x7, x5, x6	R	funct7=00000000, rs2= 00110, rs1=00101, func t3000, rd=00111, opcod e=0110011	00000000011000101000001110110011	x006283B3
eketsa x7, x 5, 12	I	rs1= 0101,fuct3= 000,r d= 00111,opcode=0010 011, imm[11:0]	00000000110000101000001110010011	ox00C2839 3
boloka x7, 8 (x5)	S	imm[11:5], rs2=00111, r s1 = 00101, funct3= 01 0, opcode= 0100011	00000000011100101010010000100011	Ox0072A42 3
lekana x5, x 6, 8	B	rs2= 0110, rs1= 00101, funct3= 000, opcode= 1 100011	00000000110001010000100011 10001 1	ox00628463

3 Design-Decision Log

(a) Register-file size. What does it cost and buy in encoding bits, forwarding-comparator width and register pressure?

LIMO uses 32 general-purpose registers. This provides more registers for keepig temporary values, variables and procedure data, reducing register pressure and the need to move values frequently between registers and memory. The cost is that each register identifier requires 5 bits, since $2^5=32$. Therefore, instructions containing rd, rs1 and rs2 require 5 bit register fields. The forwarding unit must also compare 5 bit register identifiers when detecting dependencies between instructions. A smaller register file, such as 16 registers, would require only 4-bit identifiers and narrower forwarding comparisons, but would increase register pressure. In our sample programs, 32 registers allow several working values to remain in registers without unnecessary access

(b) Architectural or microarchitectural? Which registers are programmer-visible, and which exist only inside the pipeline (IR, IF/ID, …)? Why must the ISA not specify the latter?

Architectural registers: The general purpose registers (Sephetho0 to sephetho7). The stack pointer(mokou) Microarchitectural registers: They only exist inside implementation. They are; instruction register,the pipeline registers IF/ID, IDEX, EX/MEM and MEM/WB. ISA does not specify pipeline registers as ISA, by definition, is the abstract interface between software and hardware. A program cannot see an IR or an IF/ID register, because the machine must behave as if instructions are executed one at a time in order. If the ISA fixed those registers, every implementation would be forced into the same pipeline, so a single-cycle, multicycle or deeper pipeline, LIMO could no longer run the same binaries. Leaving them out allows pipeline change without changing the ISA or breaking existing programs

(c) Branch resolution. In which stage does LIMO resolve branches, and what does that cost in flushed instructions versus extra hardware?

Limo resolves branches in the instruction decode (ID)stage. We read the two register values and do the comparison and calculate the branch target right there. Cost if branch is taken is a 1 cycle penalty, because when we are in ID figuring out the branch, we already fetched the next instruction in IF. That one was wrong, so we flush it

(d) Load-use. Why does a load followed by a dependent instruction still stall under full forwarding? Complete the diagram with your own two instructions.

If a dependent instruction follows immediately, it requires that data at the beginning of its EX stage to perform its calculation. Without a stall, its EX stage would occur during clock cycle 4.

Clock cycle	1	2	3	4	5	6	7
bofa x1, 0(x2)	IF	ID	EX	MEM	WB		
kopanya x3,x1,x4		IF	Stall(ID)	EX	MEM	WB	

(e) A flags register? RISC-V has none. Would LIMO benefit from one, and what would it do to your hazards?

No LIMO would not benefit. LIMO's trick is that it resolves branches early in ID by reading flags that were set by the previous instruction but RISC-V has no flags. It does the compare and branch in one instruction e.g lekana x1, x2, label. To know if we branch, we must run the ALU comparison first. The ALU is in the EX stage. Cost stays: I or 2 cycle branch penalty, not 0. LIMO would not cut the branch penalty.

(f) What breaks first as programs grow: immediate range, branch reach or register count?

In LIMO we use 32 bit registers and 12 bit immediates for I-type and S-type instructions, so the range is only [-2048,2047]. As programs grow, accessing stack variables, arrays, or constants beyond the range [-2048,2048] from the base register is impossible in one instruction. In my program I have to do multiple eketsa steps to reach far addresses, which increased code size. Branch reach is 21 bits and we have 32 registers, so those are enough, therefore the 12-bit immediate limit is hit first

4 Sample LIMO Programs

Program 1: sum 0 to 10

```
bofahh letsete0, 0
  bofahh sephetho0, 0
  bofahh sephetho1, 10
  bofahh nakoana1, 1

loop:
  kopanya letsete0, letsete0, sephetho0 # sum += i
  eketsa sephetho0, sephetho0, 1 # i++
  tlosa nakoana2, sephetho1, sephetho0 # temp check
  le nakoana3, letsete0, sephetho0 #
  kapa nakoana4, letsete0, nakoana1
  boloka letsete0, 8(mokou)

  ekholo sephetho1, sephetho0, loop if N >= i, continue
  lekana sephetho0, nakoana1, end

end:
  chechisa sephetho0, letsete0
  suthumetsa sephetho0, sephetho0, 0
  qhoma lehe, 0(letsibolo)
```

Expected final state: [letsete0= 55]

Program 2 $(a+b)^2$ where a=5, b=3

```
bofahh sephetho0, 5 #a = 5
  bofahh sephetho1, 3 # b = 3

  kopanya sephetho2, sephetho0, sephetho1 # c = a+b = 8

  bofahh letsete0, 0 #letsete0  = 0
  bofahh letsete1, 0 #letsete1 = i = 0
  chechisa nakoana1, sephetho2 # nakoana1 = c = 8

loop_m:
  kopanya letsete0, letsete0, sephetho2 # result += c
  eketsa letsete1, letsete1, 1 # i++

  lekana letsete1, nakoana1, done # if i == c, done
  ekholo nakoana1, letsete1, loop_m # if c >= i, loop

done:
  eketsa mokou, mokou, -16
  boloka letsete0, 0(mokou) # sp = 64
  chechisa sephetho0, letsete0 # return value = 64
  qhoma lehe, 0(letsibolo)
```

Expected final state:[sephetho0=64]

Program 3: COMPARE a=50 vs b=80

```
bofahh sephetho0, 50
```

```
bofahh sephetho1, 80

lekana sephetho0, sephetho1, EQUAL
ekholo sephetho0, sephetho1, GREATER

LESS:
 bofahh sephetho0, 2
 qhoma lehe, 0(letsibolo)

GREATER:
 bofahh sephetho0, 1
 qhoma lehe, 0(letsibolo)

EQUAL:
 bofahh sephetho0, 0
 qhoma lehe, 0(letsibolo)
```

Expected final state:[sephetho0=2]

References

https://cdn1.byjus.com/wp-content/uploads/2023/08/addressing_modes_gate_notes_16.pdf

Hennessy, D. A. (n.d.). *Computer Organization and Design: RISC-V Edition.*

Stallings, W. (2010). *Computer Organization and Architecture: Designing for Performance*. New Jersey: Pearson Education.

CS3520 Lecture Notes

AI-Use Declaration

Tool	Used for	How we checked or changed the output
Claude	Navigate through GitHub	Attempt to satisfy limo GitHub requirements

We confirm that every member can explain every part of this submission and the code behind it

.Sprint goal:N/A

Not done, and why: Not familiar with github, still learning how to utilize it.

Next sprint:

Blockers:

Sprint Note

Sprint goal: *[one sentence]*

Not done, and why: Not familiar with github, still learning how to utilize it.

Next sprint:

Blockers:

- ☐ Tag `m1` pushed; link pasted on Google Classroom
- ☐ This PDF committed as `docs/M1.pdf` and uploaded
- ☐ `\guidancefalse` set; no *[placeholders]* left
- ☐ `AI_USE.md` updated in the repository
- ☐ Every member has merged or reviewed work this sprint
- ☐ Commit SHA above matches the tagged release